Project - High Level Design

on

Chat with Technology Documents (RAG)

Course Name: Gen Ai

Institution Name: Medicaps University – Datagami Skill Based Course

Sr no	Student Name	Enrolment Number
1.	Parthiv Prajapat	EN22EL301039
2.	Pranjal Mahawar	EN22CS301729
3.	Prateeksha Shukla	EN22CS301740
4.	Raj Patel	EN22CS301781
5.	Rishabh Dev Rajput	EN22CS301803

Group Name: 06D7

Project Number: GAI-29

Industry Mentor Name: Aashruti Shah

University Mentor Name: Dr.Hemlata Patel

Academic Year: 2025-2026

Table of Contents

Chapter 1	Introduction	3
	1.1. Scope of the Document	3
	1.2. Intended Audience	3
	1.3. System Overview	3
Chapter 2	System Design	4-7
	2.1. Application Design	4
	2.2. Process Flow	4
	2.3. Information Flow	5
	2.4. Components Design	6
	2.5. Key Design Considerations	6
	2.6. API Catalogue	7
Chapter 3	Data Design	8-9
	3.1. Data Model	8
	3.2. Data Access Mechanism	8
	3.3. Data Retention Policies	9
	3.4. Data Migration	9
Chapter 4	Interfaces	10
Chapter 5	State and Session Management	11
Chapter 6	Caching	12
Chapter 7	Non-Functional Requirements	13
	7.1. Security Aspects	13
	7.2. Performance Aspects	13
Chapter 8	References	14

1. Introduction

This document presents the High-Level Design (HLD) of a Retrieval-Augmented Generation (RAG) system for interacting with technology documents. The system enables users to upload professional documents such as PDFs, manuals, and reports, and ask questions in natural language. The system retrieves relevant information from the documents and generates accurate, context-aware responses using Large Language Models (LLMs).

The solution combines Natural Language Processing (NLP), Vector Databases, and LLM APIs to ensure that responses are grounded in the uploaded documents, reducing hallucination and improving reliability.

1.1. Scope of the Document

This document covers:

System architecture and design

RAG pipeline workflow (chunking, embedding, retrieval, generation)

Data handling and storage mechanisms

API structure and communication

Deployment and scalability considerations

It focuses on high-level system behavior rather than implementation-level details.

1.2. Intended Audience

This document is intended for:

Project evaluators and faculty

Software developers and AI engineers

DevOps engineers

Stakeholders interested in system architecture

1.3. System Overview

The system allows users to:

- Upload documents
- Process documents into embeddings
- Store embeddings in a vector database
- Ask questions in natural language
- Retrieve relevant content
- Generate answers using LLM

The architecture ensures:

- Accuracy (grounded responses)
- Scalability (vector DB + APIs)
- Efficiency (fast retrieval)

2. System Design

2.1. Application Design

The system follows a layered architecture:

User Interface Layer
Chat-based UI for interaction

Backend Layer (FastAPI)
Handles APIs and request processing

RAG Pipeline Layer

Document processing

Embedding generation

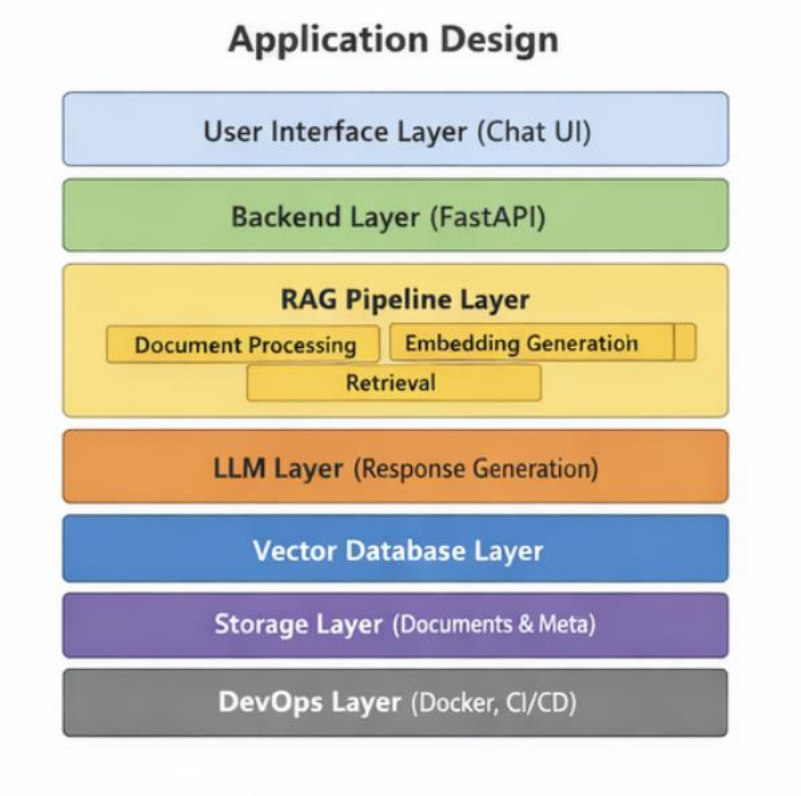
Retrieval

LLM Layer
Generates responses using context

Vector Database Layer
Stores embeddings

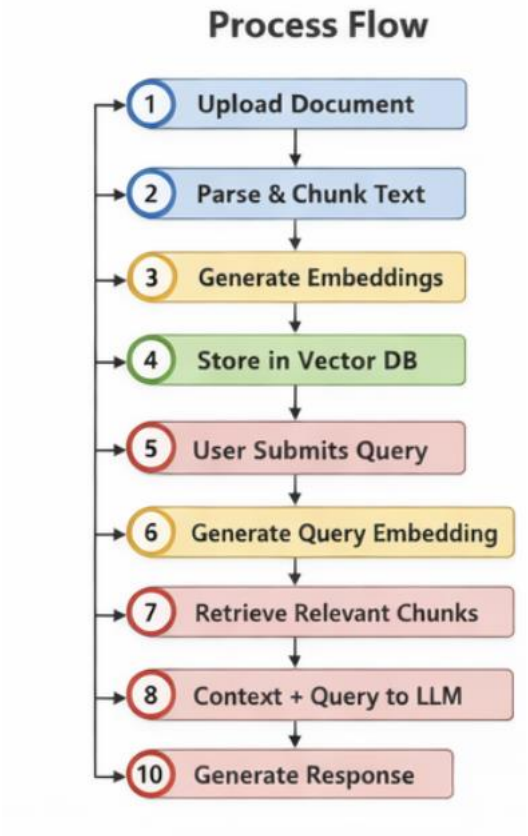
Storage Layer
Stores documents and metadata

DevOps Layer
Docker, CI/CD, deployment



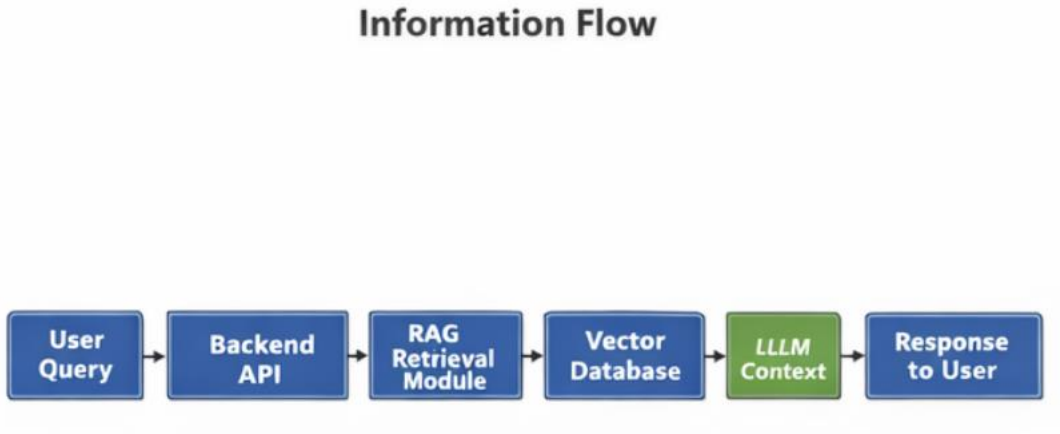
2.2. Process Flow

- User uploads document
- Document is parsed (PDF → text)
- Text is chunked into smaller parts
- Embeddings are generated
- Stored in vector database
- User asks query
- Query embedding is generated
- Similar chunks are retrieved
- Context + query sent to LLM
- Response generated and displayed



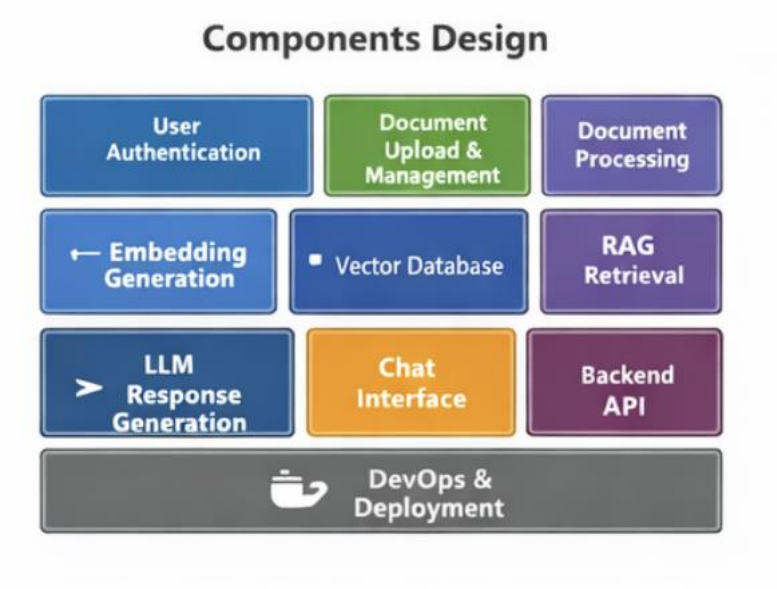
2.3. Information Flow

User Input → Backend API → RAG Retrieval Module → Vector DB → Retrieved Context → LLM → Response → User



2.4. Components Design

- User Authentication Module
- Document Upload & Management Module
- Document Processing Module
- Embedding Generation Module
- Vector Database Module
- RAG Retrieval Module
- LLM Response Generation Module
- Chat Interface Module
- Backend API Module
- Context Management Module
- Database Module
- DevOps Module



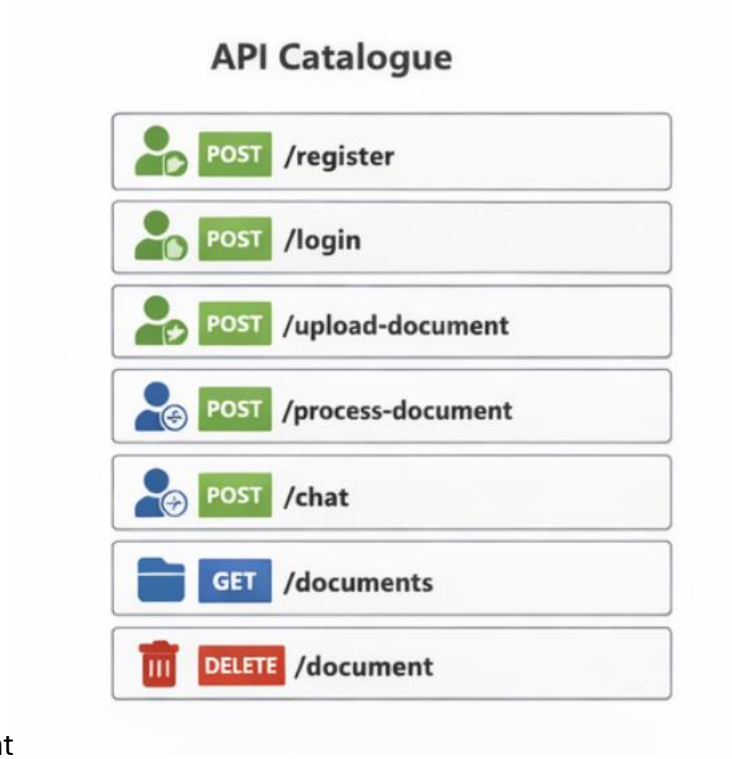
2.5. Key Design Considerations

Scalability – Handle large documents and users
Accuracy – Context-based answers (RAG)
Performance – Fast retrieval
Security – Secure data storage
Maintainability – Modular design
Cost Efficiency – Optimize API usage

2.6. API Catalogue

- POST /register – User registration
- POST /login – User login
- POST /upload-document – Upload PDF
- POST /process-document – Process & embed
- POST /chat – Ask questions
- GET /documents – List documents

- DELETE /document – Delete



document

3. Data Design

3.1. Data Model

The system uses:

User Collection

user_id

email

password

Document Collection

document_id

file_path

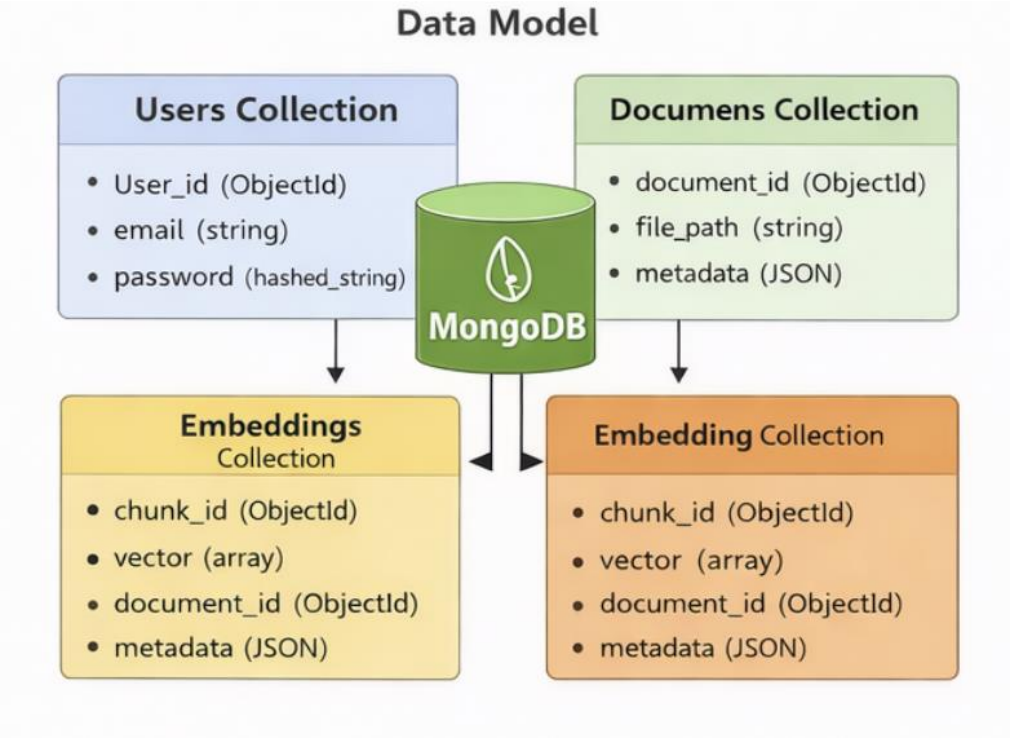
metadata

Embedding Collection

chunk_id

vector

document_id

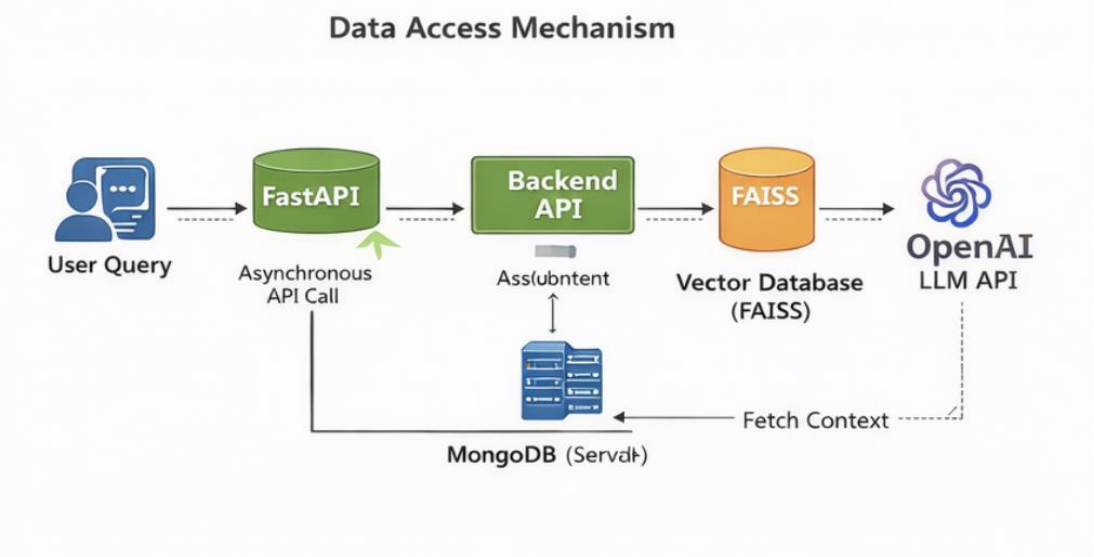


3.2. Data Access Mechanism

APIs fetch/store data

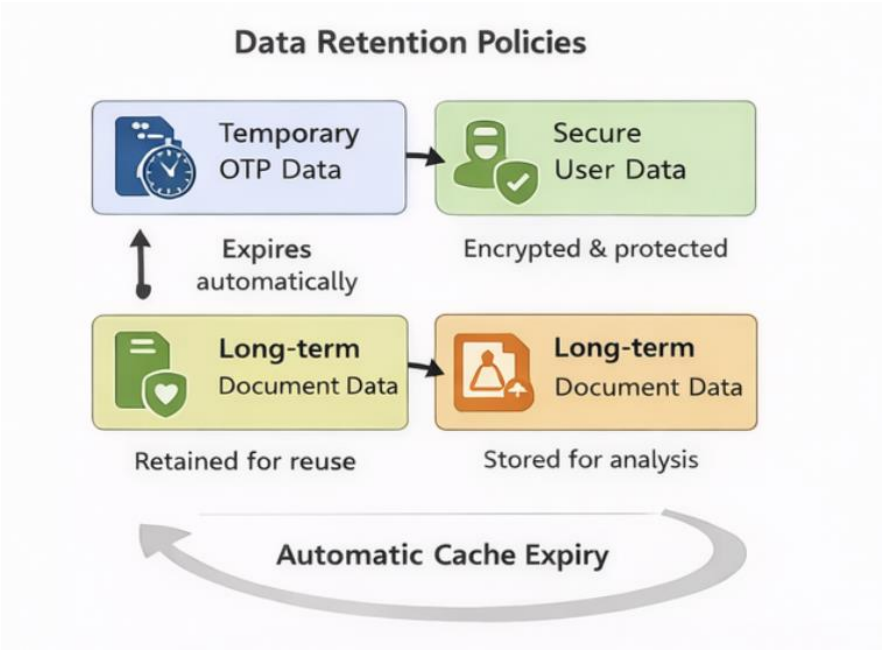
Vector DB performs similarity search

Backend handles queries asynchronously



3.3. Data Retention Policies

- Documents stored long-term
- Embeddings retained for reuse
- Logs stored for analysis
- Temporary cache auto-expired

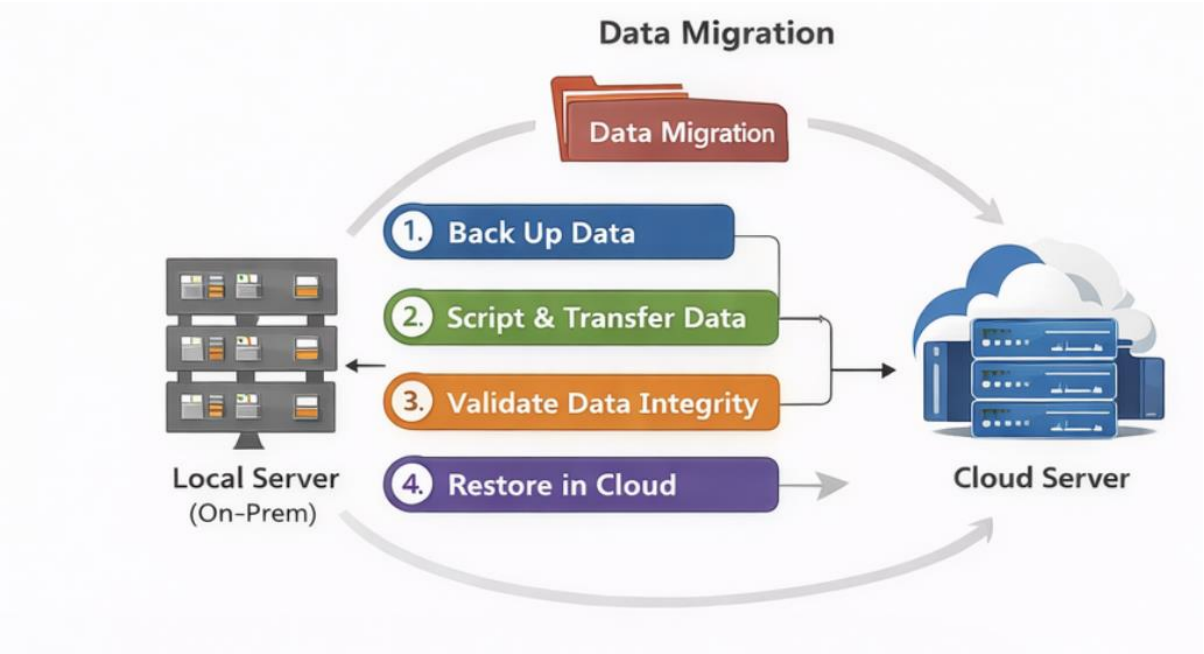


3.4. Data Migration

Local → Cloud migration

Backup & restore mechanisms

Scalable storage upgrade



4. Interfaces

The system provides well-defined interfaces to enable seamless communication between different components such as the frontend, backend, AI models, and data storage systems. These interfaces ensure modularity, scalability, and easy integration with external services.

The communication is primarily based on RESTful APIs using HTTP protocols, with data exchanged in JSON formatto ensure platform independence and interoperability.

4.1 User Interface (Frontend Interface)

The frontend provides a chat-based interface for user interaction.

Features:

User login and authentication

Document upload (PDF, manuals, reports)

Chat interface for querying documents

Display of AI-generated responses

Communication:

Sends HTTP requests to backend APIs

Receives JSON responses

Displays results in user-friendly format

Technologies (Optional):

React / HTML / CSS / JavaScript

4.2 Backend API Interface

The backend acts as the central controller of the system and exposes RESTful APIs.

Responsibilities:

Handle user requests

Route requests to appropriate modules

Coordinate between RAG pipeline, database, and LLM

Key Characteristics:

Stateless API design

JSON-based communication

Secure endpoints

5. State and Session Management

The system is designed using a stateless architecture, which ensures that each request from the client is processed independently without relying on stored session data on the server. This approach improves scalability, reliability, and ease of deployment in distributed environments.

5.1 **Stateless Architecture**

Each API request contains all required information (authentication token, query, metadata).

No session data is stored on the server between requests.

Enables horizontal scaling (multiple backend instances can handle requests).

5.2 Authentication & Session Handling

The system uses JWT (JSON Web Tokens) for authentication.

After login, a token is generated and sent to the user.

The token is included in each API request for validation.

Advantages:

No need for server-side session storage

Improved scalability

Secure user validation

5.3 **Context Management in RAG**

Although the system is stateless, context management is required for conversational AI:

Chat history is stored temporarily (in DB or cache)

Context is passed along with each query to the LLM

Enables multi-turn conversations

Example:

Query 1: "Explain vector database"

Query 2: "How does it work in this system?"
→ System uses previous context

5.4 **Session Expiry & Timeout**

JWT tokens have expiration time

Inactive sessions are automatically invalidated

Improves security and resource utilization

5.5 **Temporary Data Handling**

Temporary data includes:

Chat context

Intermediate query results

Stored in cache (Redis)

Automatically deleted after expiry

6. Caching

Caching is implemented to improve system performance, reduce latency, and minimize redundant computations in the RAG pipeline.

6.1 **Purpose of Caching**

Reduce response time

Avoid repeated LLM API calls

Decrease load on vector database

Improve user experience

6.2 **Caching Layers**

1. Query Result Caching

Frequently asked questions are cached

If same query appears → return cached response

2. Embedding Caching

Avoid regenerating embeddings for same text

Stored in cache for reuse

3. Retrieval Caching

Retrieved document chunks cached

Reduces repeated vector search

6.3 **Caching Technology**

Redis (In-Memory Data Store) is used

High-speed read/write operations

Supports TTL (Time-To-Live)

6.4 **Cache Expiration Policy**

Cache entries expire automatically

TTL based on:

Query frequency

Data freshness

Example:

Frequently used queries → longer cache

Rare queries → short cache

6.5 **Benefits**

Faster responses

Reduced API cost

Improved scalability

Lower database load

7. Non-Functional Requirements

Non-functional requirements define the system's quality attributes such as performance, security, scalability, and reliability.

7.1 Scalability

- Supports increasing number of users and documents
- Horizontal scaling using cloud infrastructure
- Vector DB optimized for large datasets

7.2 Availability

- System ensures high uptime
- Load balancing distributes traffic
- Failover mechanisms ensure continuity

7.3 Reliability

- Consistent and accurate responses
- Fault-tolerant architecture
- Retry mechanisms for API failures

7.4 Maintainability

- Modular architecture
- Easy to update components
- Clear separation of concerns

7.5 Usability

- Simple chat-based interface
- User-friendly document upload
- Fast and intuitive responses

8. References

1. FastAPI Documentation. *FastAPI – Modern, Fast Web Framework for Building APIs with Python.*
   Available at: https://fastapi.tiangolo.com/
2. MongoDB Documentation. *MongoDB Official Documentation.*
   Available at: https://www.mongodb.com/docs/

3. Docker Inc. *Docker Documentation – Containerization Platform.*
Available at: https://docs.docker.com/
4. LangChain Documentation Framework for building LLM-powered applications
Available at: https://python.langchain.com/
5. *OpenAI API Documentation– Large Language Model APIs and embeddings*
Available at: https://platform.openai.com/docs/
6. Google Generative AI Documentation. Gemini & Generative AI tools.
Available at:https://ai.google.dev/
7. FAISS (Vector Database)Facebook AI Similarity Search
Available at:https://github.com/facebookresearch/faiss
8. Google AI. *Generative AI and Large Language Models Documentation.*
Available at: https://ai.google.dev/